ELIMINACIÓN DE GAUSS

Asignatura: Computación Avanzada y Analítica — Universidad EIA
Tema: Eliminación de Gauss (Grupo 1)
Objetivo: Desarrollo matemático, implementación computacional y análisis del método para resolver sistemas de ecuaciones lineales.

Portada

- **Título:** Eliminación de Gauss: Desarrollo Matemático, Algoritmos e Implementación
- **Curso:** Computación Avanzada y Analítica
- **Institución:** Universidad EIA
- **Integrante:** Sebastian Orjuela Perez

Introducción y Contexto

¿Qué problema resuelve el método?

- Permite encontrar el vector solución x en sistemas de n ecuaciones lineales con n incógnitas expresados en la forma matricial $Ax = b$.

Idea Principal

- Transformar la matriz de coeficientes original en una **matriz triangular superior** equivalente mediante operaciones elementales de fila y, posteriormente, despejar las variables en orden inverso (Sustitución Hacia Atrás).

Importancia

- Es el método directo fundamental en álgebra lineal numérica.
- Sirve de base teórica y práctica para otros algoritmos de factorización matricial como la Factorización LU.

Desarrollo Matemático — Formulación y Equivalencia

Representación Matricial

El sistema lineal se representa algebraicamente como:

```
A * x = b

[ a11  a12  ...  a1n ]  [ x1 ]  [ b1 ]
[ a21  a22  ...  a2n ] * [ x2 ] = [ b2 ]
[ :    :    . .  :  ]  [ :  ]  [ :  ]
[ an1  an2  ...  ann ]  [ xn ]  [ bn ]
```

Matriz Aumentada

Agrupa la matriz de coeficientes y el vector de términos independientes:

```
[A | b] = [ a11  a12  ...  a1n  |  b1 ]
          [ a21  a22  ...  a2n  |  b2 ]
          [ :    :    . .  :    |  :  ]
          [ an1  an2  ...  ann  |  bn ]
```

Justificación de Operaciones Elementales de Fila

1. **Intercambio de filas (Ri <-> Rj):** Permuta renglones para evitar pivotes nulos o inestabilidad.
2. **Multiplicación por escalar no nulo (c * Ri):** Normaliza renglones según se requiera.
3. **Sustitución de fila (Ri <- Ri - m_ik * Rk):** Anula coeficientes situados por debajo del elemento pivote.

Nota de preservación: Multiplicar por matrices elementales invertibles *Ek ... E1 A x = Ek ... E1 b* preserva de forma exacta el espacio solución único del sistema.

Desarrollo Matemático — Fases del Método

1. Eliminación Hacia Adelante

- Para cada etapa *k* (columna pivote) y para cada fila *i* > *k*:
- Se calcula el factor multiplicador:
  `m_ik = a_ik / a_kk`
- Se actualizan los elementos de la fila correspondiente:
  `Ri <- Ri - m_ik * Rk`

2. Sustitución Hacia Atrás

- Para la última variable:
  `xn = bn' / ann'`
- Para las variables restantes en orden descendente (i = n-1, n-2, ..., 1):
  `xi = (bi' - SUMA(aij' * xj para j = i+1 hasta n)) / aii'`

Condiciones de Aplicación, Ventajas y Limitaciones

Condiciones de Aplicación

- La matriz *A* debe ser cuadrada (*n x n*) y no singular (det(*A*) != 0).
- Los elementos pivote *a_kk* deben ser no nulos en cada etapa del proceso de eliminación.

Ventajas

- Es un método directo: calcula la solución exacta en un número finito y predecible de pasos (a diferencia de métodos iterativos).
- Estructura algorítmica clara y altamente generalizable.

Limitaciones

- Susceptible a fallos si se presenta un pivote nulo (*a_kk = 0*) o valores muy cercanos a cero (requiere incorporar pivoteo parcial).
- Propagación de errores de redondeo en sistemas mal condicionados.
- Alta complejidad temporal en matrices densas: *O(n^3)*.

Implementación en Python

Nota: Implementación manual sin hacer uso de librerías de alto nivel como numpy.linalg.solve() o scipy.linalg.solve().

```
import numpy as np

def eliminacion_gaussiana(A, b):
    """
    Resuelve el sistema Ax = b mediante Eliminación Gaussiana simple.
    """
    A = A.astype(float)
    b = b.astype(float)
    n = len(b)

    # 1. Eliminación hacia adelante
    for k in range(n - 1):
        if A[k, k] == 0:
            raise ValueError(f"Pivote nulo detectado en la posición ({k},{k})")
        for i in range(k + 1, n):
            factor = A[i, k] / A[k, k]
            A[i, k:] -= factor * A[k, k:]
            b[i] -= factor * b[k]

    # 2. Sustitución hacia atrás
    x = np.zeros(n)
    x[n - 1] = b[n - 1] / A[n - 1, n - 1]

    for i in range(n - 2, -1, -1):
        suma = np.dot(A[i, i + 1:], x[i + 1:])
        x[i] = (b[i] - suma) / A[i, i]

    return x
```

Ejemplo Práctico Paso a Paso

Sistema a resolver:

- 2x + y - z = 8
- -3x - y + 2z = -11
- -2x + y + 2z = -3

Matriz Aumentada Inicial [A | b]

```
[ 2   1  -1 |  8 ]
[-3  -1   2 |-11 ]
[-2   1   2 |  -3 ]
```

Paso 1: Anular elementos bajo el pivote a11 = 2

- Fila 2: *R2 <- R2 - (-3/2)*R1 = R2 + 1.5*R1* ⇒ [0, 0.5, 0.5 | 1]
- Fila 3: *R3 <- R3 - (-2/2)*R1 = R3 + 1.0*R1* ⇒ [0, 2.0, 1.0 | 5]

Ejemplo Práctico — Triangulación y Solución

Paso 2: Anular elementos bajo el pivote a22 = 0.5

- Fila 3: *R3 <- R3 - (2/0.5)*R2 = R3 - 4*R2* ⇒ [0, 0, -1.0 | 1]

Matriz Triangular Superior Resultante

```
[ 2   1   -1 |  8 ]
[ 0  0.5  0.5|  1 ]
[ 0   0   -1 |  1 ]
```

Sustitución Hacia Atrás y Verificación

1. -1 * z = 1 ⇒ **z = -1**
2. 0.5 * y + 0.5 * (-1) = 1 ⇒ 0.5 * y = 1.5 ⇒ **y = 3**
3. 2 * x + 3 - (-1) = 8 ⇒ 2 * x = 4 ⇒ **x = 2**

Vector Solución Final: x = [2, 3, -1]^T

Ecuación Original	Sustitución del Vector Solución	Resultado	Estado
2x + y - z	2(2) + 3 - (-1) = 4 + 3 + 1	8	Correcto (✓)

Ecuación Original	Sustitución del Vector Solución	Resultado	Estado
-3x - y + 2z	-3(2) - 3 + 2(-1) = -6 - 3 - 2	-11	Correcto (✓)
-2x + y + 2z	-2(2) + 3 + 2(-1) = -4 + 3 - 2	-3	Correcto (✓)

Aplicación en Física e Ingeniería

Análisis de Circuitos Eléctricos mediante Leyes de Kirchhoff (KVL)

- En una red eléctrica de *n* mallas interconectadas, la aplicación de la Ley de Voltajes de Kirchhoff genera un sistema de ecuaciones lineales simultáneas expresado como: `R * I = V`
- **R:** Matriz de resistencias caracterizadas por las propiedades del circuito.
- **I:** Vector de corrientes de malla desconocidas a determinar.
- **V:** Vector de tensiones de las fuentes de alimentación presentes en cada malla.
- **Relevancia:** La Eliminación Gaussiana permite resolver analíticamente las corrientes operativas de la red para verificar el comportamiento térmico y de potencia del circuito antes de su construcción.

Conclusiones

1. **Base Numérica Fundamental:** La Eliminación de Gauss es el algoritmo directo canónico para la resolución de sistemas de ecuaciones lineales en ingeniería.
2. **Costo y Eficiencia:** Presenta una complejidad de *O(n^3)* en la fase de reducción y *O(n^2)* en la sustitución, haciendo de la eliminación la fase computacionalmente determinante.
3. **Rigor y Aplicación:** Cumple un rol crítico en modelos físicos reales como el análisis de redes eléctricas, análisis estructural y ecuaciones diferenciales discretizadas.